

Mastermind — Design

Project 2, Phase 1 · package `assignment2` · EID: ak54582

■ Reusable in another guessing game ■ Mastermind-specific □ Entry point (glue) · solid ▷ = extends dashed ▷ = implements ◆ = owns dashed → = uses/creates

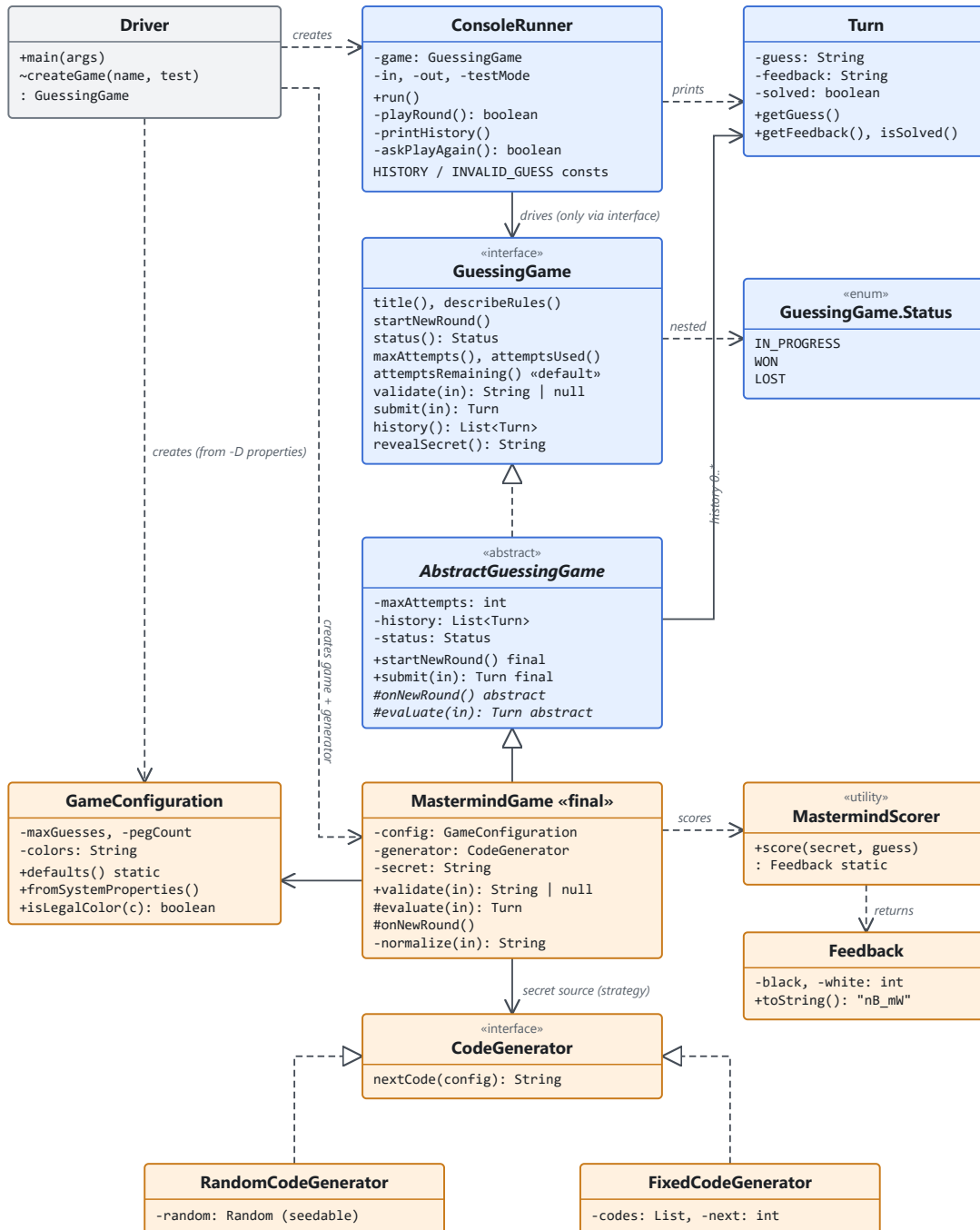


Figure 1. Classes and interfaces in `assignment2`. The console loop sits above the `GuessingGame` line and never sees a peg or a color; everything below `AbstractGuessingGame` is Mastermind.

Architecture in one paragraph

The design splits the program along one seam: **the game loop** (prompting, reporting invalid input, the HISTORY command, win/loss messages, play-again) versus **the game rules** (what a legal guess is, how it is scored, where the secret comes from). `ConsoleRunner` owns the loop and talks only to the `GuessingGame` interface. `AbstractGuessingGame` implements the bookkeeping every "N tries to guess a secret" game shares, using the template-method pattern: its `submit()` is `final` and calls the subclass hooks `validate()` and `evaluate()`. `MastermindGame` fills in those hooks. All tunable numbers live in `GameConfiguration`, and the secret comes from a swappable `CodeGenerator` strategy, so test mode and configuration changes never touch game logic.

Responsibility of each type

Type	Category	Responsibility
<code>GuessingGame</code> (interface)	Reusable	The contract between the loop and any game: title, rules text, status, attempt counts, <code>validate</code> (returns <code>null</code> or a reason), <code>submit</code> , history, <code>revealSecret</code> . Defines the nested <code>Status</code> enum (<code>IN_PROGRESS</code> , <code>WON</code> , <code>LOST</code>).
<code>AbstractGuessingGame</code>	Reusable	Implements the shared rules once: attempt limit, recording each valid <code>Turn</code> , <code>WON/LOST</code> transitions, "an invalid guess never consumes an attempt", "you cannot guess after the round ends", and resetting on <code>startNewRound()</code> . Exposes history as an unmodifiable list. Subclasses supply only <code>onNewRound()</code> and <code>evaluate()</code> (plus <code>validate()</code> from the interface).
<code>ConsoleRunner</code>	Reusable	Text UI for any <code>GuessingGame</code> : banner and rules, "Guess k of N" prompt, <code>INVALID_GUESS</code> : messages, <code>Result</code> : lines, the case-insensitive <code>HISTORY</code> command, win/loss announcement, the Y/N play-again loop, and clean exit at end of input. Input and output streams are injected, so tests can script entire sessions. All user-visible tokens are constants in one place.
<code>Turn</code>	Reusable	Immutable record of one valid turn: normalized guess text, feedback text, and whether it solved the round. Deliberately stores feedback as a <code>String</code> so it carries no Mastermind types.
<code>Driver</code>	Glue	Entry point (<code>java assignment2.Driver mastermind [test]</code>). Parses arguments, builds the configuration from <code>-D</code> properties, picks <code>RandomCodeGenerator</code> or (in test mode with <code>mastermind.secret / MASTERMIND_SECRET</code> set) <code>FixedCodeGenerator</code> , and hands the game to <code>ConsoleRunner</code> . Adding a game means adding one case to <code>createGame</code> .

MastermindGame	Mastermind	Mastermind rules: a legal guess is exactly <i>pegCount</i> legal colors; input is trimmed and case-normalized against the color set; scoring delegates to <code>MastermindScorer</code> ; a turn is solved when black pegs equal <i>pegCount</i> ; the secret comes from the injected <code>CodeGenerator</code> at each new round. Writes the rules text from the configuration.
MastermindScorer	Mastermind	Pure static function <code>score(secret, guess)</code> . Counts exact matches (black) first and removes them; each remaining secret peg can pair with at most one remaining guess peg of the same color (white = sum over colors of the smaller leftover count). This is what makes duplicate colors score correctly. No I/O, so it can be tested exhaustively.
Feedback	Mastermind	Immutable black/white value with <code>equals/hashCode</code> ; prints as <code>nB_mW</code> .
GameConfiguration	Mastermind	Immutable settings: max guesses (default 12), pegs per code (default 4), legal colors (default <code>BGOPRY</code> , 1–10 distinct visible characters). Rejects bad values in the constructor. <code>fromSystemProperties()</code> reads <code>mastermind.guesses</code> , <code>mastermind.pegs</code> , <code>mastermind.colors</code> .
CodeGenerator (interface)	Mastermind	Strategy for producing a secret: <code>nextCode(config)</code> . Swapping implementations is how test mode gets a known secret without any <code>if (testMode)</code> in the rules.
RandomCodeGenerator	Mastermind	Normal play: each peg is an independent uniform color, so repeats are possible. Accepts a seeded <code>Random</code> for reproducible runs.
FixedCodeGenerator	Mastermind	Test mode: returns a given list of secrets in order, cycling. Checks each against the active configuration (length and legal colors) and fails fast with a clear message.

What I expect to reuse in another game

Take a different guessing game, such as Wordle, Hangman, or "guess the number, higher/lower". These pieces carry over unchanged:

- **GuessingGame + Status**: the interface has no Mastermind vocabulary. Guesses and secrets are strings, and feedback is whatever text the game produces.
- **AbstractGuessingGame**: the attempt limit, history, win/loss transitions, and the "invalid input is free" rule are identical across these games. A new game writes three methods: `validate`, `evaluate`, `onNewRound`.
- **ConsoleRunner**: prompting, `HISTORY`, invalid-guess reporting, and play-again work for any implementation. Because I/O is injected, the same scripted-session test style also carries over.
- **Turn**: "guess text + feedback text + solved?" describes a turn in each of those games.
- **Driver**, with one new case in `createGame`.
- **The strategy idea behind CodeGenerator** (random in play, fixed in tests). The idea carries over; the current interface does not, because it takes a `GameConfiguration` (see below).

What is intentionally Mastermind-specific

- **MastermindScorer and Feedback**: black/white pegs and the `nB_mW` format *are* Mastermind. Generalizing them (for example, a generic "per-position match" type) would make the one tricky algorithm harder to read and test, with nothing gained.
- **MastermindGame**: the definition of a legal guess (fixed length, from a color set), case normalization against colors, and "solved when black == pegs".
- **GameConfiguration**: pegs and colors mean nothing to other games. It is kept separate from the reusable layer; `AbstractGuessingGame` only receives the `maxAttempts` integer it needs.
- **CodeGenerator, RandomCodeGenerator, FixedCodeGenerator**: they take a `GameConfiguration` and produce color strings. I chose not to make these generic (for example, `SecretSource<C>`) in Phase 1: with only one game it would be speculative. If a second game arrives, that is the first refactor I would make.

Key design decisions

- **Validation is a query, not only an exception.** `validate()` returns `null` or a readable reason, so the runner can print `INVALID_GUESS: ...` without using exceptions for control flow. `submit()` still re-validates and throws, so the rule cannot be bypassed through the API.
- **final template methods.** `submit()` and `startNewRound()` are `final`, so a subclass cannot forget to record history or to update status.
- **A constructed game is ready to play.** The constructor starts the first round, and `ConsoleRunner.run()` calls `startNewRound()` only for later rounds. This keeps the first fixed secret from being skipped; a regression test covers it.
- **Configuration without code changes.** Guesses, pegs and colors can all be changed from the command line, for example `-Dmastermind.pegs=6 -Dmastermind.colors=0123456789`. No line of game logic hard-codes 4, 12 or `BGOPRY`.